

TP 1 : gestion des identités et des accès

Module Sécurité Cloud - Durée 3 h - Travail en trinome

Objectif

Le CM de ce matin a posé un principe : dans le cloud, l'identité est le périmètre. Ce TP le met en pratique. Vous allez lire des politiques d'accès existantes et identifier ce qui ne va pas, écrire une politique de moindre privilège, remplacer une clé statique par un rôle à identifiants temporaires, puis rédiger la relation de confiance qui permet à une chaîne d'intégration continue de se passer de clés.

À la fin de la séance, vous devez savoir répondre à trois questions devant un responsable de projet : qui peut faire quoi sur cette ressource, comment le prouver, et que se passe-t-il si ces identifiants fuient.

Cadre

- Travail limité aux ressources fournies dans ce sujet et à votre poste. Toute tentative d'accès à un système tiers sort du cadre pédagogique et relève de l'article 323-1 et suivants du Code pénal.
- Les commandes de ce sujet ont été exécutées et vérifiées. Si l'une échoue, signalez-le : c'est probablement un écart d'environnement, pas une faute de votre part.

Rendu

Un dossier compressé contenant :

- le compte rendu en PDF, huit pages maximum, avec les tableaux demandés et les réponses rédigées ;
- les fichiers `lecture.json`, `confiance.json`, `confiance-ci.json` et `besoin.yml` ;
- les sorties de commande utiles, jamais de captures d'écran illisibles.

Aucun identifiant réel ne doit figurer dans le rendu, même expiré, même de l'émulateur : c'est une habitude à prendre dès maintenant.

1 Mise en place de l'environnement

Principe

Aucun compte cloud, aucune machine virtuelle, aucun droit administrateur. L'API est émulée en local par `moto`, une bibliothèque Python installée dans un environnement virtuel, c'est-à-dire dans votre dossier personnel. Moto est une bibliothèque Python incontournable qui permet de mimer (mock) l'ensemble des services d'Amazon Web Services (AWS) localement. Elle évite d'envoyer de vraies requêtes à AWS pendant les phases de test ou de développement, ce qui fait gagner du temps et de l'argent. Le serveur n'écoute que sur `127.0.0.1` : il n'est pas exposé sur le réseau et ne déclenche pas de demande d'autorisation du pare-feu. Prérequis unique : **Python 3.10 ou**

supérieur. Vérifiez-le avant tout avec `python3 --version` (Linux, macOS) ou `py -3 --version` (Windows). Sous macOS, si la commande déclenche une proposition d'installation des outils en ligne de commande, acceptez-la, ou installez Python depuis python.org.

Vous pouvez évidemment faire usage d'Internet.

Étape 1 : créer l'environnement et installer les outils

Ouvrez un terminal et placez-vous dans votre dossier de travail.

Linux et macOS

```
python3 -m venv tp1-iam
source tp1-iam/bin/activate
pip install "moto[server]" awscli parliament policy_sentry "setuptools<81"
```

Windows, dans l'invite de commandes `cmd.exe` :

```
py -3 -m venv tp1-iam
tp1-iam\Scripts\activate.bat
pip install "moto[server]" awscli parliament policy_sentry "setuptools<81"
```

Si vous préférez PowerShell (je ne recommande pas...), l'activation est `tp1-iam\Scripts\Activate.ps1`, précédée si besoin de `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`, qui ne vaut que pour la fenêtre en cours et ne demande aucun droit administrateur.

Le paquet `setuptools` n'est pas décoratif : sans lui, l'analyseur `parliament` s'arrête sur l'erreur `No module named 'pkg_resources'`. Il affiche par ailleurs un avertissement de dépréciation à chaque exécution, sans conséquence sur les résultats.

Étape 2 : démarrer le serveur

Toujours dans ce terminal, lancez le serveur et **laissez la fenêtre ouverte** pendant toute la séance :

```
moto_server -p 5055
```

La fenêtre doit afficher `Running on http://127.0.0.1:5055`. Le port 5055 est choisi à dessein : le port 5000, souvent proposé dans la documentation, est fréquemment déjà pris, notamment par le récepteur AirPlay sur macOS.

Étape 3 : préparer le second terminal

Ouvrez un **second** terminal, placez-vous dans le même dossier et activez de nouveau l'environnement, avec la commande d'activation de l'étape 1. Déclarez ensuite quatre variables. Les identifiants sont factices, l'émulateur les accepte tels quels.

Linux et macOS

```
export AWS_ACCESS_KEY_ID=test
export AWS_SECRET_ACCESS_KEY=test
export AWS_DEFAULT_REGION=eu-west-3
export AWS_ENDPOINT_URL=http://localhost:5055
```

Windows, `cmd.exe` :

```
set AWS_ACCESS_KEY_ID=test
set AWS_SECRET_ACCESS_KEY=test
set AWS_DEFAULT_REGION=eu-west-3
set AWS_ENDPOINT_URL=http://localhost:5055
```

Windows, PowerShell :

```
$env:AWS_ACCESS_KEY_ID="test"
$env:AWS_SECRET_ACCESS_KEY="test"
$env:AWS_DEFAULT_REGION="eu-west-3"
$env:AWS_ENDPOINT_URL="http://localhost:5055"
```

C'est la seule différence entre les trois systèmes. Grâce à la variable `AWS_ENDPOINT_URL`, **toutes les commandes du sujet s'écrivent ensuite exactement de la même façon partout**, sans option supplémentaire.

Étape 4 : vérifier

```
aws sts get-caller-identity
```

Vous devez obtenir un bloc JSON contenant `"Account": "123456789012"`. Deux diagnostics possibles en cas d'échec :

- `Could not connect to the endpoint URL` : le serveur de l'étape 2 n'est pas démarré, ou la fenêtre a été fermée ;
- `command not found` ou `aws` n'est pas reconnu : l'environnement virtuel n'est pas activé dans ce terminal.

Conventions du sujet

- Tous les fichiers JSON et YAML sont donnés en clair et sont à créer avec votre éditeur de texte, dans le dossier courant. Aucune commande du sujet ne les génère à votre place.
- Amazon Resource Name (ARN) : c'est l'identifiant unique et mondial d'une ressource AWS.
- Le numéro de compte de l'émulateur est toujours `123456789012`. Les identifiants de ressources (ARN) du sujet sont donc utilisables tels quels, sans les recopier depuis une sortie précédente.
- Chaque commande tient sur une seule ligne.

Ce que l'émulateur fait, et ce qu'il ne fait pas

À lire avant de commencer : L'émulateur reproduit fidèlement les API : création d'utilisateurs, de politiques, de rôles, endossement de rôle, délivrance d'identifiants temporaires avec date d'expiration. En revanche, il **n'évalue pas** les politiques : un appel non autorisé aboutira quand même, et aucun refus ne sera jamais affiché.

Ce n'est pas une limite gênante pour ce TP, à condition d'en tirer la bonne conclusion : ici, une politique se juge en la lisant et en l'analysant, pas en constatant un refus. C'est d'ailleurs la situation réelle d'un ingénieur en revue d'architecture, qui doit se prononcer sur une politique avant tout déploiement. Les exercices 2, 3 et 5 reposent donc sur l'analyse statique et sur votre raisonnement.

2 Lire et critiquer des politiques existantes

Les trois politiques de l'annexe A sont extraites, sous forme simplifiée, de situations réelles. Créez un fichier par politique (`a.json`, `b.json`, `c.json`).

1. Avant tout outil, lisez chaque politique et notez ce qui vous paraît anormal. Cette étape n'est pas facultative : l'outil ne remplace pas la lecture.
2. Passez ensuite l'analyseur statique :

```
parliament --file a.json
parliament --file a.json --include-community-auditors --minimum_severity HIGH
```

Deux points de fonctionnement à connaître. L'outil affiche à chaque exécution un avertissement sur `pkg_resources` : ignorez-le. Et il refuse d'être redirigé vers un fichier ou vers un tube, avec le message `You cannot pass a file with --file and use stdin together` : lisez ses résultats à l'écran et recopiez-les dans votre compte rendu.

- Attention : l'une des trois politiques ne produit *aucun* constat avec la première commande, et plusieurs constats graves avec la seconde. Ce n'est pas une panne de votre installation, c'est le sujet de la question 4.
- Pour chaque politique, remplissez le tableau suivant dans votre compte rendu.

Fichier	Constats à la lecture	Constats de l'outil	Correction proposée
a.json			
b.json			
c.json			

- Question à traiter en deux ou trois phrases : l'outil a-t-il vu des choses que vous n'aviez pas vues, et inversement ? Que concluez-vous sur l'usage d'un analyseur en revue de code ?

3 Écrire une politique de moindre privilège

Besoin exprimé par l'équipe applicative : *le service de reporting doit lire les objets du préfixe 2026/ du bucket acme-reports, et écrire uniquement dans 2026/entrant/. Rien d'autre.*

- Générez une première version avec `policy_sentry`, qui traduit un besoin exprimé en niveaux d'accès : Créez d'abord le fichier `besoin.yml` avec ce contenu :

```
mode: crud
read:
  - "arn:aws:s3:::acme-reports/2026/*"
write:
  - "arn:aws:s3:::acme-reports/2026/entrant/*"
```

puis lancez :

```
policy_sentry write-policy --input-file besoin.yml
```

- Lisez la politique affichée. L'outil est volontairement large : il inclut toutes les actions de la catégorie demandée. Combien d'actions comporte-t-elle ? Lesquelles sont inutiles pour ce besoin précis ? Recopiez-la dans un fichier `genere.json` si vous souhaitez la conserver pour le compte rendu.
- Produisez `lecture.json`, votre version resserrée : les seules actions réellement nécessaires, et deux conditions, l'une imposant le transport chiffré, l'autre restreignant la plage d'adresses source. Le modèle attendu est celui vu en cours :

```
"Condition": {
  "Bool": { "aws:SecureTransport": "true" },
  "IpAddress": { "aws:SourceIp": "203.0.113.0/24" }
}
```

- Vérifiez que votre version ne produit aucun constat :

```
parliament --file lecture.json
```

Une sortie vide signifie aucun constat. Si vous obtenez `No resources match for the given action`, relisez la forme de vos ARN : une action portant sur des objets exige un ARN se terminant par `/*`.

- Créez la politique et rattachez-la à un utilisateur :

```
aws iam create-user --user-name analyste
aws iam create-policy --policy-name LectureRapports2026 --policy-document file://lecture.json
aws iam attach-user-policy --user-name analyste --policy-arn arn:aws:iam::123456789012:policy/LectureRapports2026
aws iam list-attached-user-policies --user-name analyste
```

4 Du secret permanent au rôle temporaire

1. Créez une clé d'accès statique pour `analyste` et observez sa forme :

```
aws iam create-access-key --user-name analyste
aws iam list-access-keys --user-name analyste
```

Notez l'identifiant obtenu et sa date de création. Quelle est sa date d'expiration ?

2. Créez maintenant un rôle endossable par cet utilisateur. Le premier document est la *relation de confiance* : il dit *qui* peut endosser le rôle, et non ce qu'il pourra faire. Créez le fichier `confiance.json` :

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "AWS": "arn:aws:iam::123456789012:user/analyste" },
    "Action": "sts:AssumeRole"
  }]
}
```

puis créez le rôle et rattachez-lui la politique de l'exercice précédent :

```
aws iam create-role --role-name RoleLecture --assume-role-policy-document file://confiance.json
aws iam attach-role-policy --role-name RoleLecture --policy-arn arn:aws:iam::123456789012:policy/LectureRapports2026
```

3. Endossez le rôle et comparez :

```
aws sts assume-role --role-arn arn:aws:iam::123456789012:role/RoleLecture --role-session-name tp1 --duration-seconds 900
```

Trois éléments sont à relever dans votre compte rendu : le préfixe de l'identifiant délivré et ce qui le distingue de celui de la question 1, la présence d'un jeton de session, et la date d'expiration.

4. Supprimez la clé statique, puisqu'elle n'a plus lieu d'être :

```
aws iam delete-access-key --user-name analyste --access-key-id AKIA...
aws iam list-access-keys --user-name analyste
```

5. Rédigez en quelques lignes : un attaquant récupère les deux jeux d'identifiants dans un dépôt Git public, six mois après leur création. Que peut-il faire avec chacun ? Reliez votre réponse à l'incident Capital One vu en cours.

5 Une chaîne d'intégration continue sans clés

Objectif : un pipeline GitHub Actions doit déployer sans qu'aucune clé ne soit stockée dans la forge. Le mécanisme est la fédération : le pipeline présente un jeton signé par son fournisseur d'identité, et l'échange contre un rôle.

1. Rédigez `confiance-ci.json`, relation de confiance d'un rôle endossable par le seul dépôt `acme/rapports`, sur la seule branche `main`. Quatre éléments sont à assembler :
 - l'action `sts:AssumeRoleWithWebIdentity`;
 - un principal fédéré : un ARN de type `oidc-provider` sur le compte `123456789012`, désignant le domaine `token.actions.githubusercontent.com`;
 - une condition `StringEquals` fixant l'audience à `sts.amazonaws.com`;
 - une condition `StringLike` sur le sujet, dont la valeur est `repo:acme/rapports:ref:refs/heads/main`.
2. Créez le rôle correspondant et rattachez-lui la politique de l'exercice 2 (il s'agit de la partie 3 du TP) .
3. Question de sécurité : que se passe-t-il si la condition sur le sujet est omise, ou écrite `repo:acme/*`? Formulez la réponse en termes d'attaquant, pas en termes de configuration.
4. Passez l'analyseur sur ce fichier :

```
parliament --file confiance-ci.json
```

Vous obtiendrez deux constats, `Statement contains neither Resource nor NotResource` et `Unknown federation source`. Expliquez pourquoi tous deux sont ici des faux positifs. C'est le deuxième enseignement du TP sur l'outillage : un analyseur a un domaine de validité, ici les politiques d'identité et un catalogue fini de fournisseurs fédérés.

6 Synthèse : la grille d'audit

Reprenez la grille de l'annexe B, issue de la synthèse du cours. Pour chaque ligne, indiquez si l'environnement que vous venez de construire la respecte, et ce qu'il faudrait ajouter pour les lignes non couvertes.

Plusieurs lignes ne peuvent pas être satisfaites ici, soit parce que l'émulateur n'implémente pas le mécanisme, soit parce que la question ne se pose que dans une organisation réelle. Identifiez-en au moins trois et dites, pour chacune, ce qu'il faudrait mettre en place dans un vrai compte et comment vous en apporteriez la preuve à un auditeur.

Annexe A : politiques à auditer

a.json

```
{
  "Version": "2012-10-17",
  "Statement": [
    { "Sid": "AccesEquipe", "Effect": "Allow", "Action": "*", "Resource": "*" }
  ]
}
```

b.json

```
{
  "Version": "2012-10-17",
  "Statement": [
    { "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject"],
      "Resource": "arn:aws:s3:::acme-reports" },
    { "Effect": "Allow",
      "Action": "iam:PassRole",
      "Resource": "*" }
  ]
}
```

c.json

```
{
  "Version": "2012-10-17",
  "Statement": [
    { "Sid": "DeploiementApplicatif",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:ListBucket", "iam:CreateAccessKey"],
      "Resource": ["arn:aws:s3:::acme-reports",
                   "arn:aws:s3:::acme-reports/*",
                   "*"] }
  ]
}
```

Annexe B : grille d’audit

Point de contrôle	Respecté ?	Justification
Compte racine scellé, sous MFA matériel, inutilisé au quotidien		
Authentification fédérée et MFA résistant au phishing pour les humains		
Rôles et identifiants temporaires pour les humains, les workloads et la CI		
Politiques versionnées, revues et testées		
Recertification périodique des droits		
Environnements séparés par comptes distincts		
Aucune action ou ressource en * hors cas encadré		
Aucune clé d’accès statique		
Aucun compte partagé ni compte local non fédéré		
Aucun secret dans le code ou les tickets		

Annexe C : mémo des commandes

```
# Environnement (terminal 1, a laisser tourner)
moto_server -p 5055
# Terminal 2 : activer l'environnement, puis declarer les 4 variables (voir section 1)

# Utilisateurs, politiques
aws iam create-user --user-name NOM
aws iam create-policy --policy-name NOM --policy-document file://f.json
aws iam attach-user-policy --user-name NOM --policy-arn arn:aws:iam::123456789012:policy/NOM
aws iam list-attached-user-policies --user-name NOM

# Cles statiques
aws iam create-access-key --user-name NOM
aws iam list-access-keys --user-name NOM
aws iam delete-access-key --user-name NOM --access-key-id AKIA...

# Roles et identifiants temporaires
aws iam create-role --role-name NOM --assume-role-policy-document file://f.json
aws iam attach-role-policy --role-name NOM --policy-arn arn:aws:iam::123456789012:policy/NOM
aws sts assume-role --role-arn arn:aws:iam::123456789012:role/NOM --role-session-name tp1 --
    duration-seconds 900

# Analyse statique
parliament --file f.json
parliament --file f.json --include-community-auditors --minimum_severity HIGH
policy_sentry write-policy --input-file besoin.yml
```